



Pontifícia Universidade Católica de Minas Gerais

Graduação em Sistema de Informação

Disciplina: Sistemas Operacionais

Professor: Lucas Bragança da Silva

Alunos: Marlon Magalhães Carvalho,

Danton Rodrigues Diniz

Introdução

A memória virtual é uma técnica utilizada pelos sistemas operacionais para permitir que programas utilizem espaços de endereçamento maiores que a memória física disponível. A tradução de endereços lógicos para endereços físicos é realizada utilizando tabelas de páginas e memórias associativas denominadas TLBs.

O objetivo deste trabalho foi implementar um simulador de memória virtual em linguagem C, contemplando a tradução de endereços, tratamento de faltas de página, substituição de páginas e estatísticas de desempenho.

Desenvolvimento

O sistema foi dividido em cinco módulos:

- main.c
- page_table.c
- memory.c
- tlb.c
- statistics.c

A tabela de páginas possui 256 entradas e a memória física contém 128 quadros.

Quando uma página não é encontrada ocorre uma falta de página. A página é carregada do arquivo BACKING_STORE.bin utilizando fseek e fread.

O TLB foi implementado utilizando a política FIFO.

A substituição de páginas utiliza o algoritmo Aging, que aproxima o comportamento do algoritmo LRU por meio de bits de referência e contadores de envelhecimento.

Resultados

Foram executados dois conjuntos de testes.

Teste 1 – Endereços aleatórios

- Number of Translated Addresses = 10000
- Page Faults = 4958
- Page Fault Rate = 0.496
- TLB Hits = 654
- TLB Hit Rate = 0.065

```
w64devkit
Logical address: 65034 Physical address: 25354 Value: 10
Logical address: 24921 Physical address: 26457 Value: 89
Logical address: 27194 Physical address: 12858 Value: 58
Logical address: 62093 Physical address: 30861 Value: -115
Logical address: 46151 Physical address: 22087 Value: 71
Logical address: 46725 Physical address: 3973 Value: -123
Logical address: 18071 Physical address: 11927 Value: -105
Logical address: 8802 Physical address: 17506 Value: 98
Logical address: 57524 Physical address: 27828 Value: -76
Logical address: 56034 Physical address: 7138 Value: -30
Logical address: 49298 Physical address: 28306 Value: -110
Logical address: 9288 Physical address: 17224 Value: 72
Logical address: 10242 Physical address: 25090 Value: 2
Logical address: 16671 Physical address: 26399 Value: 31
Logical address: 39089 Physical address: 22961 Value: -79
Logical address: 13226 Physical address: 11946 Value: -86
Logical address: 15456 Physical address: 17504 Value: 96
Logical address: 65367 Physical address: 16215 Value: 87
Logical address: 30221 Physical address: 20749 Value: 13
Logical address: 6747 Physical address: 3419 Value: 91
Logical address: 43374 Physical address: 29806 Value: 110
Logical address: 2954 Physical address: 17290 Value: -118
Logical address: 29362 Physical address: 25266 Value: -78
Logical address: 37997 Physical address: 10861 Value: 109
Logical address: 32949 Physical address: 11957 Value: -75
Logical address: 51173 Physical address: 19173 Value: -27
Logical address: 46911 Physical address: 7231 Value: 63
Logical address: 8906 Physical address: 17610 Value: -54
Logical address: 20691 Physical address: 3539 Value: -45
Logical address: 37757 Physical address: 13181 Value: 125
Logical address: 9801 Physical address: 17225 Value: 73
Logical address: 31640 Physical address: 26520 Value: -104
Logical address: 40925 Physical address: 27613 Value: -35
Logical address: 52326 Physical address: 18022 Value: 102
Logical address: 7424 Physical address: 12800 Value: 0
Logical address: 14532 Physical address: 25284 Value: -60
Logical address: 61732 Physical address: 29988 Value: 36
Logical address: 34478 Physical address: 17582 Value: -82
Logical address: 3929 Physical address: 3417 Value: 89
Logical address: 4230 Physical address: 17286 Value: -122
Logical address: 44308 Physical address: 28692 Value: 20
Logical address: 55234 Physical address: 30658 Value: -62
Logical address: 34194 Physical address: 7826 Value: -110
Number of Translated Addresses = 10000
Page Faults = 4958
Page Fault Rate = 0.496
TLB Hits = 654
TLB Hit Rate = 0.065
~/Downloads/PROJETO/vm manager $
```

Teste 2 - Localidade de referência:

- Page Fault Rate: 11,6%
- TLB Hit Rate: 79,2%

Os resultados demonstram a importância da localidade de referência para o desempenho da memória virtual.

```
w64devkit
Logical address: 41425 Physical address: 6865 Value: -47
Logical address: 25899 Physical address: 22571 Value: 43
Logical address: 41074 Physical address: 9330 Value: 114
Logical address: 40786 Physical address: 30802 Value: 82
Logical address: 40325 Physical address: 25477 Value: -123
Logical address: 26823 Physical address: 26823 Value: -57
Logical address: 40540 Physical address: 23388 Value: 92
Logical address: 45352 Physical address: 15400 Value: 40
Logical address: 40772 Physical address: 30788 Value: 68
Logical address: 40253 Physical address: 25405 Value: 61
Logical address: 41104 Physical address: 9360 Value: -112
Logical address: 41221 Physical address: 6661 Value: 5
Logical address: 41159 Physical address: 9415 Value: -57
Logical address: 40238 Physical address: 25390 Value: 46
Logical address: 41433 Physical address: 6873 Value: -39
Logical address: 40968 Physical address: 9224 Value: 8
Logical address: 40881 Physical address: 30897 Value: -79
Logical address: 40962 Physical address: 9218 Value: 2
Logical address: 40536 Physical address: 23384 Value: 88
Logical address: 40403 Physical address: 25555 Value: -45
Logical address: 40917 Physical address: 30933 Value: -43
Logical address: 40628 Physical address: 23476 Value: -76
Logical address: 41151 Physical address: 9407 Value: -65
Logical address: 41364 Physical address: 6804 Value: -108
Logical address: 41374 Physical address: 6814 Value: -98
Logical address: 40344 Physical address: 25496 Value: -104
Logical address: 40303 Physical address: 25455 Value: 111
Logical address: 20976 Physical address: 22768 Value: -16
Logical address: 40303 Physical address: 25455 Value: 111
Logical address: 16797 Physical address: 15005 Value: -99
Logical address: 23011 Physical address: 26851 Value: -29
Logical address: 41065 Physical address: 9321 Value: 105
Logical address: 42561 Physical address: 19265 Value: 65
Logical address: 30126 Physical address: 3246 Value: -82
Logical address: 40251 Physical address: 25403 Value: 59
Logical address: 41293 Physical address: 6733 Value: 77
Logical address: 40449 Physical address: 23297 Value: 1
Logical address: 56232 Physical address: 32168 Value: -88
Logical address: 41031 Physical address: 9287 Value: 71
Logical address: 40363 Physical address: 25515 Value: -85
Logical address: 40244 Physical address: 25396 Value: 52
Logical address: 40488 Physical address: 23336 Value: 40
Logical address: 40975 Physical address: 9231 Value: 15
Number of Translated Addresses = 10000
Page Faults = 1164
Page Fault Rate = 0.116
TLB Hits = 7925
TLB Hit Rate = 0.792
~/Downloads/PROJETO/vm_manager $
```

Conclusão

O simulador implementou corretamente os mecanismos de memória virtual propostos. O uso do TLB reduziu o tempo de tradução dos endereços e o algoritmo Aging apresentou comportamento próximo ao LRU.

Os resultados obtidos confirmam os conceitos teóricos de paginação por demanda, substituição de páginas e localidade de referência.

Uso de Inteligência Artificial

A IA foi utilizada para:

- Explicação dos conceitos.
- Depuração do código.
- Correção e implementação de functions incompletas.
- Geração de casos de teste.
- Análise dos resultados.

Os prompts seguiram a metodologia Spec-Driven Development, especificando requisitos funcionais, restrições de implementação e critérios de aceitação.

Todo o código produzido foi analisado, compreendido e validado pelo grupo antes da entrega.

PROMPTS NO PADRÃO SDD

Obs: Além do padrão SDD, eu gosto sempre de adicionar uma mensagem no final do meu prompt, a fim de evitar que a IA alucine e me de algo que eu não queira, mesmo que ela ache melhor e alucine, ao invés de sair alterando todo meu código, ela vai validar comigo e explicar o porquê da sugestão dela fora do escopo do meu prompt é bom. Assim, aprendo com ela e descarto os códigos que não quero e que ela alucinou.

Os prompts abaixo representam os principais usos de IA durante o desenvolvimento. Algumas interações ocorreram em formato de tutoria guiada, depuração e revisão incremental; por isso, os prompts foram consolidados por tema no formato Spec-Driven Development, mantendo fidelidade às solicitações feitas durante o trabalho.

Prompt 1

Implemente a função `tlb_lookup` em C seguindo os requisitos, restrições, critérios de aceitação abaixo. Não implemente nada fora do que está pedido abaixo, caso tenha alguma solução melhor ou alguma sugestão, não altere o código, primeiro me mostre o que acha que seria melhor e o porquê: Aí eu vou validar se eu quero ou não a sua sugestão.

Requisitos funcionais:

- Procurar uma página em um TLB de 16 entradas.
- Retornar o frame se encontrado.
- Retornar -1 caso contrário.

Requisitos não funcionais:

- Complexidade $O(n)$.
- Não utilizar memória dinâmica.

Restrições:

- Linguagem C11.
- Utilize a estrutura `tlb_entry_t` existente.
- Não altere a interface da função.

Critério de aceitação:

- Página encontrada retorna o frame correto.
- Página inexistente retorna -1.

Prompt 2

Implemente o algoritmo Aging para aproximação do LRU. Não implemente nada fora do que está pedido abaixo, caso tenha alguma solução melhor ou alguma sugestão, não altere o código, primeiro me mostre o que acha que seria melhor e o porquê: Aí eu vou validar se eu quero ou não a sua sugestão.

Requisitos:

- Contador de 8 bits.
- Deslocar o contador a direita.
- Inserir o bit R no MSB.
- Limpar o bit R.

Restrições:

- C11.
- Não alterar a estrutura `page_table_entry_t`.

Critério:

- Páginas recentemente acessadas devem possuir maior contador.

Prompt 3

Contexto:

Estou desenvolvendo um simulador de memória virtual em C. O programa traduz endereços lógicos para físicos usando TLB, tabela de páginas e memória física. Quando uma página não está carregada, ocorre page fault e a página deve ser lida do arquivo BACKING_STORE.bin.

Problema:

Preciso corrigir e entender a função `handle_page_fault`, responsável por carregar uma página ausente na memória física.

Requisitos funcionais:

- Validar se o número da página está dentro do intervalo permitido.
- Procurar um frame livre.
- Caso não exista frame livre, selecionar uma página vítima.
- Verificar se a vítima encontrada é válida.
- Obter o frame da página vítima.
- Invalidar a página vítima na tabela de páginas.
- Remover a página vítima da TLB.
- Posicionar o arquivo BACKING_STORE.bin na posição `page * PAGE_SIZE` usando `fseek`.
- Ler `PAGE_SIZE` bytes usando `fread`.
- Verificar se `fseek` e `fread` foram executados corretamente.
- Atualizar `frame_to_page`.
- Atualizar a tabela de páginas.
- Retornar o frame onde a página foi carregada.

Requisitos não funcionais:

- Código claro e fácil de explicar.
- Evitar warnings no make.
- Evitar acessos inválidos à memória.

Restrições:

- Usar linguagem C11.
- Não usar `malloc`.
- Não alterar a assinatura da função.
- Não alterar os arquivos de cabeçalho sem necessidade.
- Usar `fseek` e `fread` para leitura do BACKING_STORE.bin.
- Não implementar funcionalidades fora do escopo da função.

Critérios de aceitação:

- O projeto deve compilar com make sem erros e sem warnings.
- A função deve tratar page fault corretamente.
- O programa deve executar com `addresses_random.txt` e `addresses_location.txt`.
- A TLB e a tabela de páginas devem ser atualizadas corretamente após substituição.
- O retorno de `fread` não pode ser ignorado.
- O programa não deve usar frame inválido.

Instrução adicional:

Não me entregue a função inteira pronta de imediato. Explique a lógica por etapas, faça perguntas para eu tentar implementar e revise meu código depois.

Prompt 4

Contexto:

Estou desenvolvendo um simulador de memória virtual em C. O programa utiliza TLB, tabela de páginas, memória física e arquivo BACKING_STORE.bin para traduzir endereços lógicos em endereços físicos.

Problema:

Preciso implementar e/ou revisar a lógica da TLB com política FIFO, garantindo busca, inserção, atualização e remoção de entradas.

Requisitos funcionais:

- Procurar uma página no TLB.
- Retornar o frame correspondente quando a página for encontrada.
- Retornar -1 quando a página não estiver no TLB.
- Inserir novas entradas no TLB.
- Atualizar o frame caso a página já exista no TLB.
- Usar política FIFO quando o TLB estiver cheio.
- Remover ou invalidar uma entrada do TLB quando uma página for substituída.

Requisitos não funcionais:

- Código simples e legível.
- Complexidade $O(n)$, considerando o tamanho reduzido do TLB.
- Evitar alterações desnecessárias no projeto.

Restrições:

- Linguagem C11.
- Não utilizar memória dinâmica.
- Não alterar a assinatura das funções.
- Não alterar a estrutura `tlb_entry_t`.
- Não implementar nada fora do escopo da TLB.

CrITÉrios de aceitação:

- Página existente no TLB deve retornar o frame correto.
- Página inexistente deve retornar -1.
- Inserções devem respeitar o limite de 16 entradas.
- Quando o TLB estiver cheio, a substituição deve seguir FIFO.
- Entradas removidas não devem ser utilizadas em traduções futuras.

Instrução adicional:

Não entregue o projeto inteiro pronto. Explique a lógica passo a passo e, caso identifique uma solução melhor fora do escopo, explique primeiro e aguarde validação.

Prompt 5

Contexto:

Estou desenvolvendo um simulador de memória virtual em C. O projeto utiliza uma tabela

de páginas com 256 entradas para mapear páginas virtuais para quadros da memória física.

Problema:

Preciso implementar e revisar as funções da tabela de páginas, incluindo consulta, atualização, invalidação, marcação de referência e apoio ao algoritmo Aging.

Requisitos funcionais:

- Verificar se o número da página está dentro dos limites da tabela.
- Retornar o frame quando a página estiver válida.
- Retornar -1 quando a página estiver inválida ou fora do intervalo.
- Atualizar frame, bit de validade, bit de referência e aging_counter.
- Invalidar completamente uma página removida da memória.
- Marcar o bit de referência quando uma página válida for acessada.
- Atualizar o aging_counter das páginas válidas.

Requisitos não funcionais:

- Código claro e fácil de explicar.
- Evitar acessos fora dos limites do vetor.
- Manter a separação entre o módulo page_table.c e os demais módulos.

Restrições:

- Linguagem C11.
- Não utilizar malloc.
- Não alterar as assinaturas das funções.
- Não alterar a estrutura page_table_entry_t.
- Não acessar a tabela de páginas diretamente fora do módulo page_table.c.

Critérios de aceitação:

- Página válida deve retornar o frame correto.
- Página inválida deve retornar -1.
- Página fora do intervalo não deve acessar posições inválidas da tabela.
- Página invalidada deve limpar frame, valid, reference_bit e aging_counter.
- O bit de referência só deve ser marcado em páginas válidas.

Instrução adicional:

Explique a lógica antes de sugerir código. Quero implementar função por função e entender o motivo de cada verificação.

Prompt 6

Contexto:

Estou desenvolvendo a política de substituição de páginas de um simulador de memória

virtual. O trabalho exige o uso do Aging Algorithm para aproximar o comportamento do LRU.

Problema:

Preciso implementar e entender a atualização dos contadores de envelhecimento das páginas residentes na memória.

Requisitos funcionais:

- Utilizar um contador de envelhecimento de 8 bits.
- Atualizar apenas páginas válidas.
- Deslocar o aging_counter uma posição para a direita.
- Inserir o bit de referência no bit mais significativo do contador.
- Zerar o bit de referência após a atualização.
- Fazer com que páginas acessadas recentemente tenham maior contador.

Requisitos não funcionais:

- Código simples e compatível com o restante do projeto.
- Evitar uso de timestamps ou histórico completo de acessos.
- Manter a lógica eficiente para uma tabela de 256 páginas.

Restrições:

- Linguagem C11.
- Não utilizar memória dinâmica.
- Não alterar a estrutura page_table_entry_t.
- Não alterar a interface das funções existentes.

Critérios de aceitação:

- Páginas acessadas recentemente devem receber valor maior no aging_counter.
- Páginas não acessadas devem ter seu contador reduzido ao longo do tempo.
- O bit de referência deve ser zerado após a atualização.
- A lógica deve aproximar o comportamento do LRU.

Instrução adicional:

Explique o raciocínio do algoritmo antes da implementação e evite entregar o código completo sem explicação.

Prompt 7

Contexto:

Estou implementando a política de substituição de páginas em um simulador de memória virtual. Quando não há quadros livres, o programa precisa escolher uma página residente para ser removida.

Problema:

Preciso implementar a função select_victim_page, que deve escolher a página válida com menor aging_counter.

Requisitos funcionais:

- Percorrer todas as páginas da tabela de páginas.
- Considerar apenas páginas válidas.
- Obter o aging_counter de cada página válida.
- Escolher como vítima a página com menor aging_counter.
- Retornar -1 caso nenhuma página válida seja encontrada.

Requisitos não funcionais:

- Código simples e fácil de explicar.
- Comparar todas as páginas válidas antes de retornar.
- Manter a lógica compatível com o Aging Algorithm.

Restrições:

- Linguagem C11.
- Não utilizar malloc.
- Não acessar diretamente a tabela de páginas fora do módulo `page_table.c`.
- Usar as funções auxiliares `page_table_is_valid` e `page_table_get_aging_counter`.

Critérios de aceitação:

- A função deve retornar uma página válida quando houver páginas residentes.
- A vítima escolhida deve ser a página com menor `aging_counter`.
- A função não deve retornar antes de comparar todas as páginas.
- Se não houver página válida, deve retornar -1.

Instrução adicional:

Não entregue a função pronta de imediato. Explique a lógica usando `victim_page`, menor contador e comparação entre páginas, para que eu consiga implementar.

Prompt 8

Contexto:

Estou corrigindo a função de tratamento de page fault em um simulador de memória virtual em C. A função precisa ler páginas do arquivo `BACKING_STORE.bin` usando `fseek` e

fread.

Problema:

Preciso validar corretamente as chamadas fseek e fread para garantir que a página correta seja lida do arquivo.

Requisitos funcionais:

- Posicionar o arquivo BACKING_STORE.bin na posição $\text{page} * \text{PAGE_SIZE}$.
- Verificar se fseek retornou erro.
- Ler PAGE_SIZE bytes para physical_memory[frame].
- Guardar o retorno de fread.
- Verificar se a quantidade lida é igual a PAGE_SIZE.
- Encerrar o programa com mensagem de erro caso fseek ou fread falhem.

Requisitos não funcionais:

- Evitar warnings do compilador.
- Evitar leitura incompleta de páginas.
- Manter o código legível e fácil de explicar.

Restrições:

- Linguagem C11.
- Usar funções padrão de I/O da linguagem C.
- Não utilizar malloc.
- Não continuar a execução caso a leitura da página falhe.
- Não alterar a assinatura da função handle_page_fault.

Critérios de aceitação:

- O código deve compilar sem warning de retorno ignorado do fread.
- O programa deve verificar se fseek retornou 0.
- O programa deve verificar se fread leu exatamente PAGE_SIZE bytes.
- Em caso de falha, deve imprimir uma mensagem de erro e encerrar.

Instrução adicional:

Explique a lógica de validação antes de mostrar qualquer trecho de código. Quero entender por que o retorno de fseek e fread precisa ser verificado.

Prompt 9

Contexto:

Estou compilando um simulador de memória virtual em C usando make. O compilador apresentou warnings relacionados à função handle_page_fault.

Problema:

Preciso entender e corrigir warnings do compilador sem alterar a lógica geral do projeto.

Requisitos funcionais:

- Identificar warning de variável frame possivelmente não inicializada.
- Identificar warning de retorno ignorado de fread.
- Explicar por que esses warnings podem indicar problemas lógicos.
- Orientar quais validações precisam ser adicionadas.
- Garantir que a função handle_page_fault continue retornando um frame válido.

Requisitos não funcionais:

- Manter o código simples.
- Evitar alterações desnecessárias.
- Priorizar segurança e legibilidade.

Restrições:

- Linguagem C11.
- Não reescrever o projeto inteiro.
- Não alterar interfaces sem necessidade.
- Explicar a causa lógica antes da correção.

Critérios de aceitação:

- O projeto deve compilar com make sem warnings.
- A variável frame deve ser inicializada antes de ser usada.
- O retorno de fread deve ser verificado.
- O programa deve continuar executando corretamente com os arquivos de teste.

Instrução adicional:

Não apenas corrija o código. Explique o motivo do warning e o que ele revela sobre a lógica do programa.

Prompt 10

Contexto:

Estou testando um simulador de memória virtual em C com dois arquivos de endereços: addresses_random.txt e addresses_location.txt.

Problema:

Preciso interpretar os resultados obtidos, principalmente Page Fault Rate e TLB Hit Rate, e explicar a diferença entre os dois testes no relatório técnico.

Requisitos funcionais:

- Comparar os resultados do arquivo de endereços aleatórios.
- Comparar os resultados do arquivo com localidade de referência.
- Explicar a diferença entre Page Fault Rate nos dois testes.
- Explicar a diferença entre TLB Hit Rate nos dois testes.
- Relacionar os resultados com localidade de referência, paginação por demanda e TLB.

Requisitos não funcionais:

- Explicação clara para relatório acadêmico.
- Não inventar resultados.
- Usar apenas os valores obtidos na execução do programa.

Restrições:

- Não alterar o código.
- Não gerar resultados fictícios.
- Considerar apenas os testes executados com os arquivos fornecidos pelo projeto.

Critérios de aceitação:

- A análise deve explicar por que o teste com localidade apresenta maior TLB Hit Rate.

- A análise deve explicar por que o teste com localidade apresenta menor Page Fault Rate.
- A explicação deve conectar os resultados aos conceitos de memória virtual.

Instrução adicional:

Explique de forma objetiva, mas suficiente para demonstrar entendimento dos conceitos envolvidos.

Prompt 11

Contexto:

Estou preparando o relatório técnico de um trabalho de Sistemas Operacionais sobre um simulador de memória virtual em C.

Problema:

Preciso revisar se o relatório atende aos critérios do enunciado, incluindo introdução, desenvolvimento, resultados, conclusão, uso de IA e link do repositório GitHub.

Requisitos funcionais:

- Verificar se o relatório possui introdução ao tema.
- Verificar se descreve a arquitetura e os módulos do projeto.
- Verificar se descreve as estruturas de dados utilizadas.
- Verificar se apresenta decisões de projeto.
- Verificar se apresenta testes realizados e análise dos resultados.

- Verificar se possui conclusão.
- Verificar se possui seção específica sobre uso de IA.
- Verificar se os prompts estão documentados no padrão SDD.
- Verificar se há link para o repositório GitHub público.

Requisitos não funcionais:

- Melhorar clareza e organização do texto.
- Corrigir erros de português.
- Manter linguagem acadêmica.
- Evitar alterar o sentido técnico do relatório.

Restrições:

- Não inventar informações.
- Usar apenas os dados reais do projeto.
- Manter coerência com o enunciado do trabalho.
- Preservar os resultados reais dos testes.

Critérios de aceitação:

- O relatório deve cobrir os itens obrigatórios do enunciado.
- A seção de IA deve documentar prompts e metodologia SDD.
- Os resultados devem ser apresentados de forma clara.
- O texto deve estar adequado para entrega em PDF.

Instrução adicional:

Aponte o que falta, explique o motivo e sugira correções antes de gerar uma versão final.

O passo a passo para a execução está no Readme do repositório do Github.

Repositório GitHub: <https://github.com/MarlonMagalhaesC/virtual-memory-manager>

“A mente que se abre a uma nova ideia jamais voltará ao seu tamanho original. ”

- Albert Einstein